



University of St.Gallen

Institute of Computer Science

Initial Event Log Analysis

Process Mining

Session 2



From insight to impact.

Outline

Session 1: Introduction

- *Event Logs, Event Log Inspection, First Exploration*

Session 2: Initial Log Analysis

- *Schema Analysis, Visualizations, Log Enrichments*

Session 3: Process KPIs

- *Process Outcomes, Dashboards*

Session 4: Process Discovery and –Modeling

- *Variants, Process Maps, BPMN, Petri Nets*

First part of Assignment 1 due

Session 5: Conformance Checking

- *Business Rules and Normative Process Models*

Session 6: Performance Mining

- *Load and Demand, Bottlenecks*

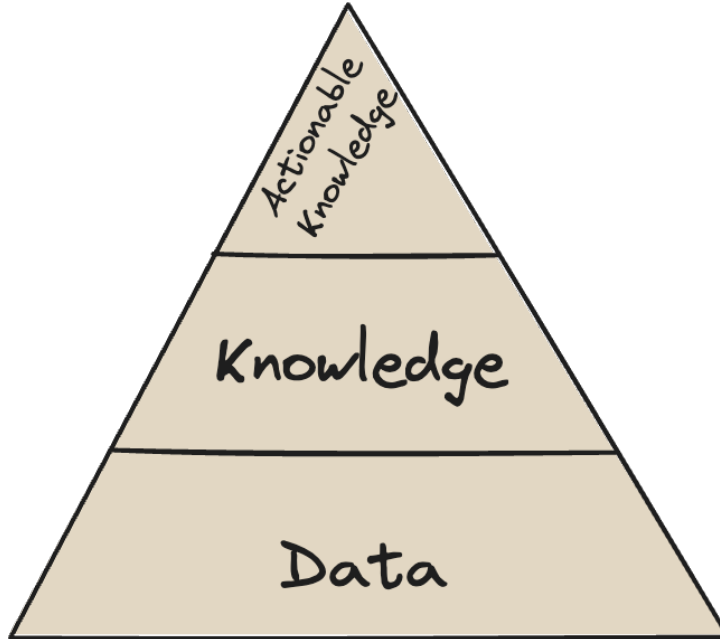
Session 7: Deviance Diagnostics

- *Explanations and Rule Mining*

Assignment 1 due

Data Analysis

cf. DIKW model



15% of cases did not meet the performance target of 21 days because of rework

Create Purchase Requisition was followed by Analyze Purchase Requisition in 382 out of 608 cases

Fjodor Kowalski created a purchase requisition at 21.01.2011 09:59:00, which took 13 mins

Two Approaches for Data Analysis

Top-Down Data Analysis

Find **answers** for given questions. (*directed*) Start with business-relevant question and decompose.

e.g.,

- What percentage of cases are paid in time? → 52%

Here we start with concrete questions and find the answers to them

Bottom-Up Data Analysis

Extract knowledge from data and already established insights to **find new questions**. (*playful*)

e.g.,

- Some cases contain multiple payments. → *Why?*

In particular:

- **Exploratory Data Analysis:** Ask questions about the data. Visualise using a set of generic tools, answer, refine and iterate.
- **Initial Data Analysis:** Check missing values, characteristics of raw data.

Here we do not start with concrete questions: data that we do not understand will lead to questions

Exploratory Data Analysis

= look at the data before making any assumptions

Outline

1. Overview
2. Activity Schema Analysis
3. Case Inspection
4. Univariate and Bivariate Analysis
5. The Case Log
6. Log Enrichment

Overview 1 – Standard Exploratory Data Analysis

Univariate Analysis

- measures of central tendency (mean, median)
- measures of dispersion (variance, standard deviation, range)
- summary through quantiles/percentiles
- **histograms**
- box plots, violin plots
- density plots

understanding a single column in a table, looking at one variable

Bivariate Analysis (*relationship between two variables*)

- **scatter plots**
- correlation matrices
- pair plots / scatter matrix

Multivariate Analysis (*relationship between three or more variables*)

- heatmaps
- 3D scatter plots
- interactive plots, animations

Overview 2 – Initial Analysis Specific to Event Logs

Disco

- Process Map
- Log Statistics
- Case Inspection

Python Basic

- Time zone and time granularity
- Schema Analysis
- Case Inspection
- The Case Log

Python Advanced

- Log Enrichment

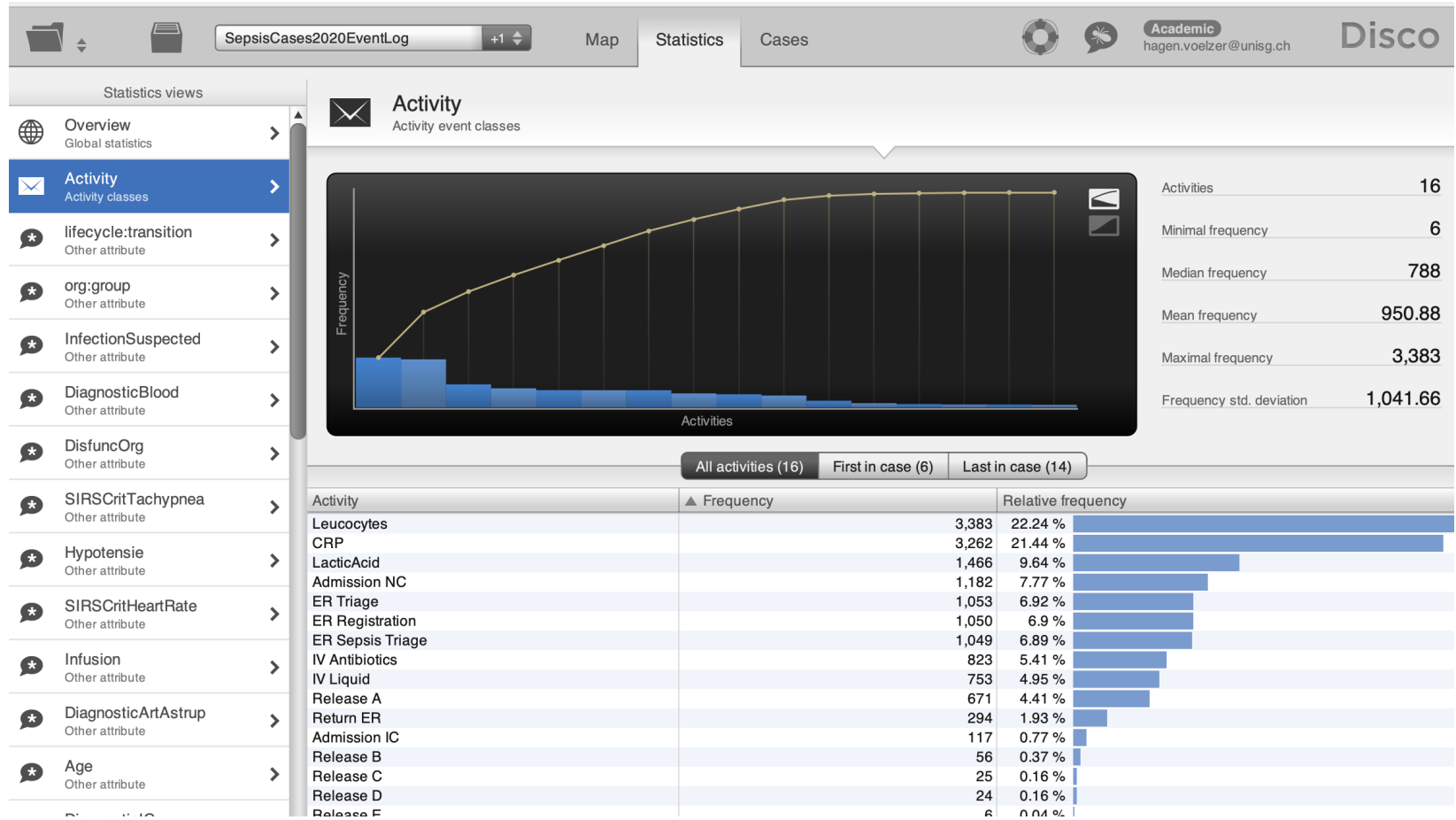
Example Event Log: Sepsis

Download [here](#). Open the log (xes file) in *Disco*.

This real-life event log contains events of a *sepsis triage and treatment* process from a hospital. Sepsis is a life-threatening condition typically caused by an infection.

One case represents the pathway through the hospital.

The events were recorded by the ERP (Enterprise Resource Planning) system of the hospital.

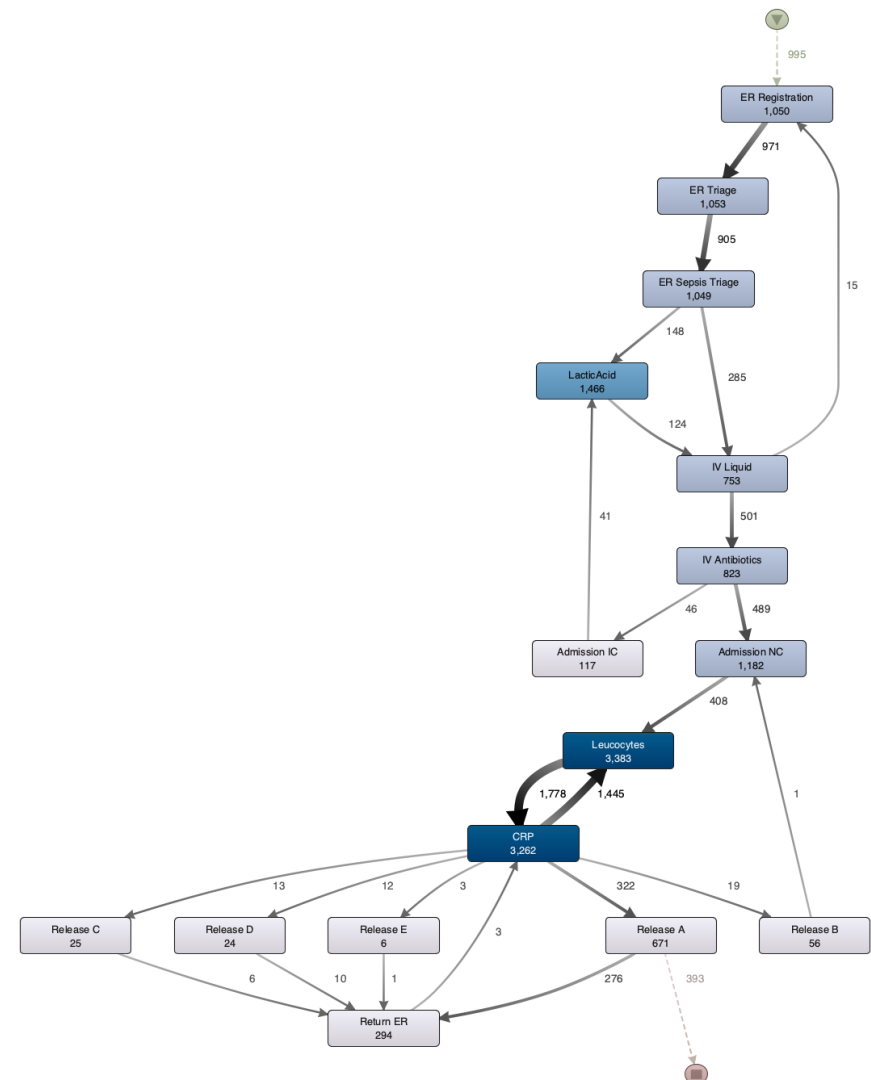


Process Map (a.k.a. Directly-Follows Graph)

Natural entry point for a new data set

- shows process activities
- their ordering in the log
- their frequencies

We will discuss the process map in detail in Session 4.



pandas_intro_sepsis.py *(intro/ recap pandas basics, offline exercise)*

Important concepts:

- Filtering
- Projection
- Aggregation
- Enrichment
- Imputation

Pandas Introduction with the Sepsis Log

This notebook recaps some basic concepts of pandas. It is not intended to be complete or a reference. For extended information, you can consult the following resources:

- [Pandas Cheat Sheet](#)
- [Pandas Documentation](#)
- [Pandas Tutorials](#)

Loading

A pandas *dataframe* can be loaded from any csv file. A dataframe is similar to a table in a database or MS Excel.

```
1 _file_name = '../data/Purchasing-Example.csv'
2 df = pd.read_csv(_file_name)
3 df
```

Search...

Columns Visualize Explore Export

Complete Timestamp

Activity

Resource

Role

Bootstrapping Checklist

- ☐ Check datatypes
 - ☐ check adequacy of data type: *Boolean, Categorical, Numerical (Integer, Float), Timestamp*
 - ☐ convert when necessary
- ☐ Check numerical granularity
 - ☐ use rounding when necessary, e.g. money as integer or fixed precision 2 decimals (*after each arithmetic operation*)
- ☐ Select mandatory attributes: *Case Id, Completion Time, Activity*
- ☐ Check time zone and time granularity
 - ☐ *set time zone first* before inspecting granularity
- ☐ Observe ordering of events within case, within log (*relevant for case inspection and sequential enrichment*)
- ☐ Select time frame
 - ☐ work with smaller intervals for fast results or to avoid memory limitations (e.g., browser related)
- ☐ Check for redundant columns and duplicate rows

Initial Event Log Analysis

This notebook integrates a set of patterns for initial event log analysis that are useful for exploring the raw event log data before doing any specific process mining analysis such as behavioral, conformance, performance, or deviance analysis.

Log Initialization

Initialization Summary

Time Frame

Granularity Analysis

Ordering Info

Initial Timestamp attributes

Select a time zone to apply to all timestamp columns in the event log and case log:

Select Time Zone UTC ▾

(Optional) Select a granularity level to apply to all timestamp columns in the event log and case log:

Select granularity level -- ▴ ▾

☐ Enforce global ordering of the event log by completion time

Select mandatory attributes from attributes that are fully filled:

CASE ID column case:concept:name ▾

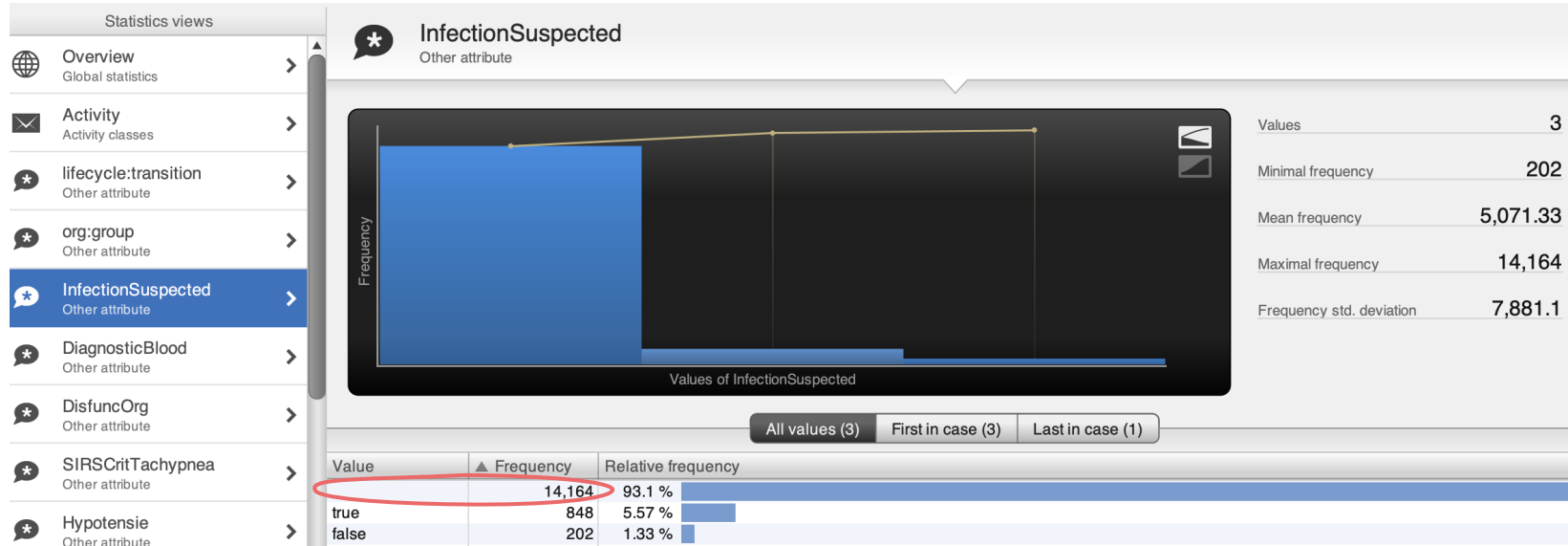
COMPLETION TIME column time:timestamp ▾

ACTIVITY column concept:name ▾

Activity Schema Analysis

= understand data attributes for each activity

Problem (*Disco* limitation)



Most attributes pertain only to one activity.

- e.g. *InfectionSuspected* pertains to *ER Registration* only
- Disco does not show here whether any values are missing for *ER Registration*

Event Log Structure

Event Log Characteristics

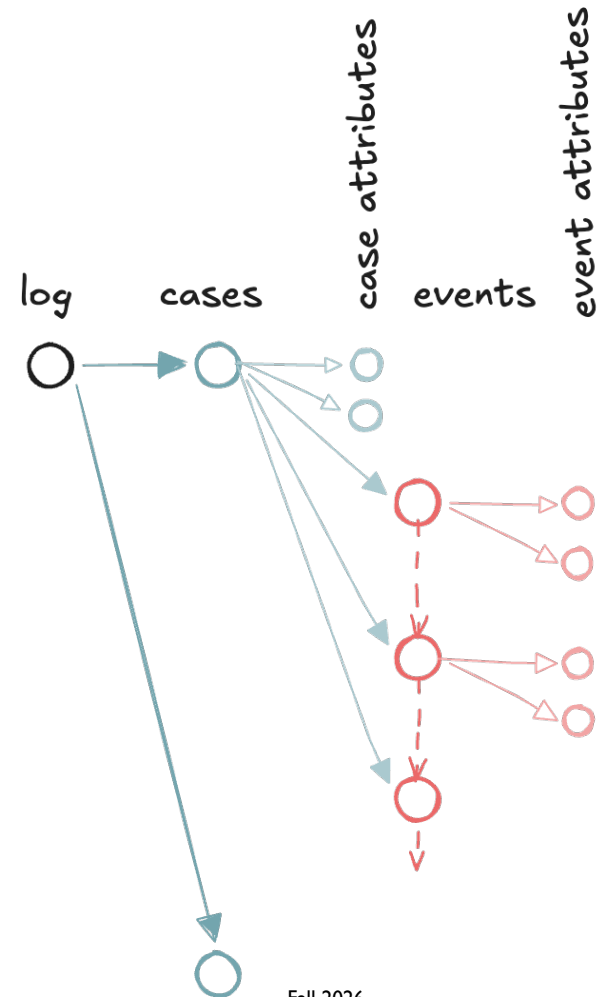
- mix of tabular, hierarchical, and sequential aspects

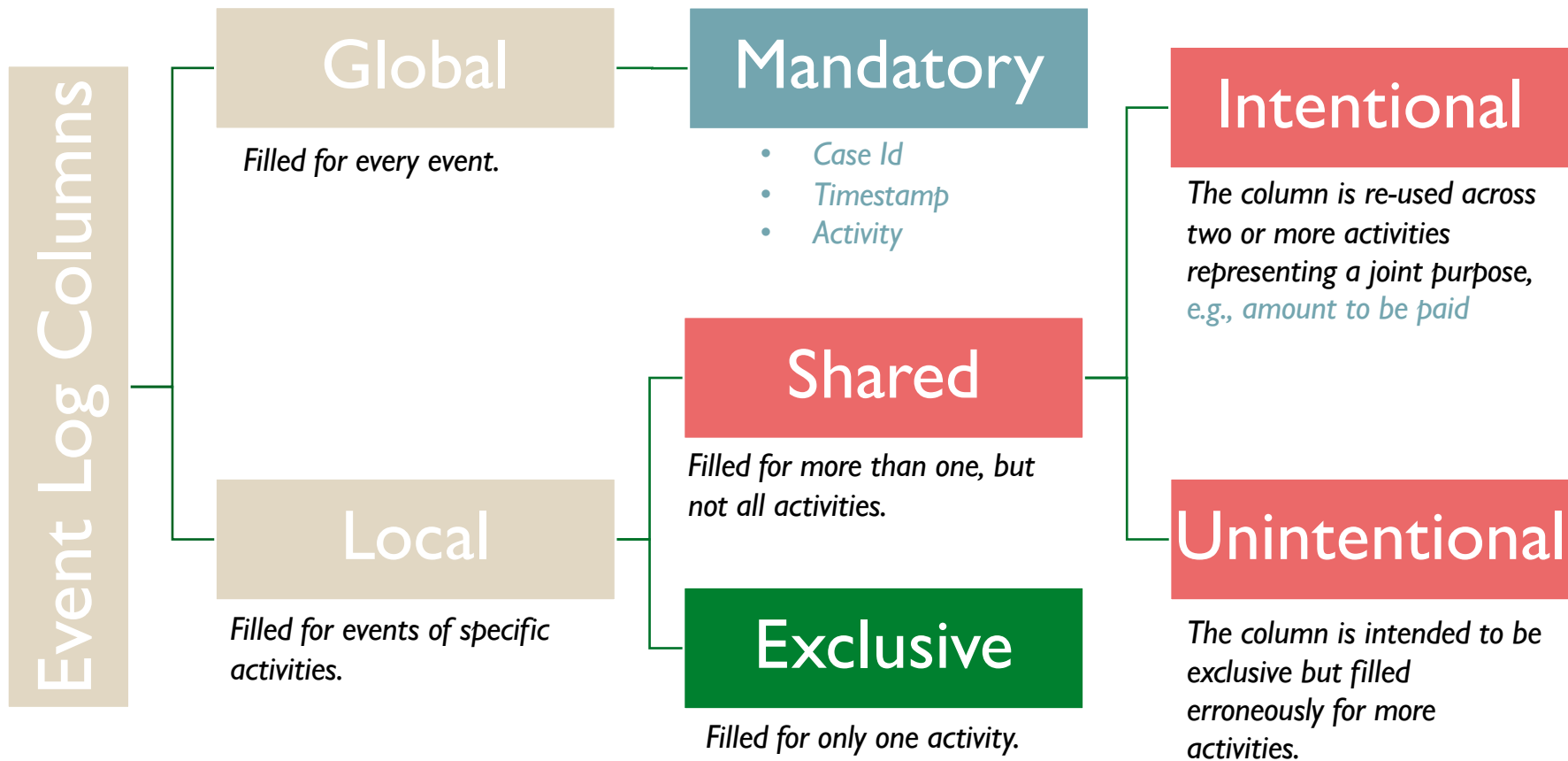
Log =

- set of cases, each case being a
 - set or **sequence** of events

Event has

- mandatory attributes (*tabular*) and
- activity specific attributes (*activity schema*)





Numeric Columns

Cumulative

~ an aggregated number,
e.g., the sum of all payment
amounts so far

Incremental

~ a standalone number, e.g.,
an individual payment
amount

Case-Cumulative

~ cumulates along the
events of the same case only,
e.g., cumulative payments for
the same loan

Log-Cumulative

~ cumulates along the events of more
than one case, e.g., cumulative
payments from all customers this
month

Activity Attributes

```
graph LR; A[Activity Attributes] --- B[Incomplete]; A --- C[Complete];
```

Incomplete

Some events of the given activity have no value for the given attribute.

Complete

Each event of the given activity has a value for the given attribute.

Activity Schemas View

A *schema* of a given activity is the set of all attributes such that at least one event of the activity has a value for that attribute. Therefore, each *global* attribute is in each schema. *Local* attributes can be *shared* between the schema of two or more activities or *exclusive* to one activity schema.

Activity Schemas Schema Overlaps

Q Search...

concept:name
str

incidence
int64


local_attributes
object
unique:

missing_values
object
unique:

concept:name str	incidence int64 	local_attributes object unique:	missing_values object unique:
Leucocytes	3,382	["Age","Diagnose","Leucocytes"]	{"Leucocytes":-22,"Diagnose":-3366,"Age":-3364}
CRP	3,262	["Age","CRP","Diagnose"]	{"CRP":-139,"Diagnose":-3257,"Age":-3252}
LacticAcid	1,466	["LacticAcid"]	{"LacticAcid":-12}
Admission NC	1,182		{}
ER Triage	1,053	["Age","Diagnose"]	{"Diagnose":-1049,"Age":-1047}
ER Registration	1,050	["Age","Diagnose","DiagnosticArtAstrup","DiagnosticBl...	{"Diagnose":-296,"Age":-55}
ER Sepsis Triage	1,049	["Age","Diagnose"]	{"Diagnose":-1044,"Age":-1042}
IV Antibiotics	823		{}
IV Liquid	753	["Age","Diagnose"]	{"Diagnose":-740,"Age":-739}
Release A	671		{}

16 rows, 3 columns

10 / page << < Page 1 of 2 > >>

Exercise: Initial Data Analysis

- What activities exist?
- What schema does each activity have?
 - *ER Registration* – Which attributes of the schema are incomplete, i.e., have missing values?
 - *Leucocytes* Does the schema of this activity have only the *Leucocytes* attribute?
 - *CRP* Does the schema of this activity have only the *CRP* attribute?
- Which attributes are shared across activity schemas?
- Classify the *Leucocytes* and *CRP* attribute as incremental or cumulative.

Cleanup

Consider cleaning up the event log:

- fix columns unintentionally filled, e.g. *Age*
- rename columns named like activities: *CRP*, *Leucocytes*
- rename columns if the name does not indicate that it is cumulative

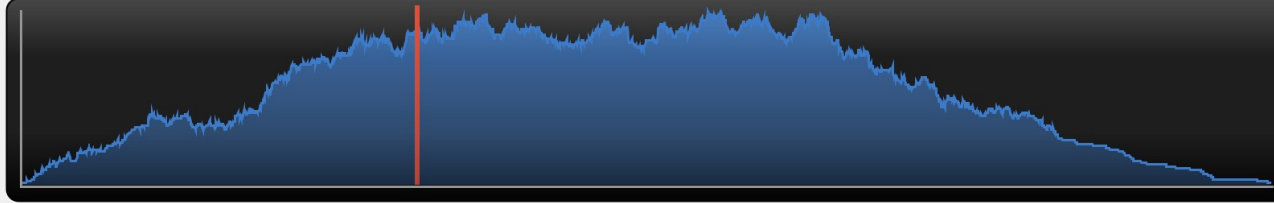
Case Sampling and Inspection

= looking at all, or selected data of a sampled case



F

Case with 10 events



Events 10

Start 05/07/2014 18:56:57

Duration 1 day, 16 hours

Graph

Table

	Activity	Date	Time	lifecycle:transition	org:group	InfectionSuspected	DiagnosticBlood	DisfuncOrg	SIRSCritTachypnea	Hypotensie	SIRSCritHeartRate	Infusion
1	ER Registration	05/07/2014	18:56:57	complete	A	true	true	false	false	false	true	true
2	ER Triage	05/07/2014	18:57:49	complete	C							
3	ER Sepsis Triage	05/07/2014	18:58:08	complete	A							
4	Leucocytes	05/07/2014	19:11:00	complete	B							
5	CRP	05/07/2014	19:11:00	complete	B							
6	LacticAcid	05/07/2014	19:11:00	complete	B							
7	IV Antibiotics	05/07/2014	22:01:28	complete	A							
8	IV Liquid	05/07/2014	22:01:28	complete	A							
9	Admission NC	05/07/2014	22:03:00	complete	I							
10	Release A	05/09/2014	11:00:00	complete	E							

empty, because either

- attribute not in *schema of activity* (cf. below), or
- value is missing

Case Inspection (Python)

```
case_data = event_log[event_log['case:concept:name']=='F']  
df = case_data[['time:timestamp', 'case:concept:name', 'concept:name', 'CRP', 'Leucocytes']]  
display(df)
```

	time:timestamp	case:concept:name	concept:name	CRP	Leucocytes
69	2014-05-07 18:56:57+00:00	F	ER Registration	NaN	NaN
70	2014-05-07 18:57:49+00:00	F	ER Triage	NaN	NaN
71	2014-05-07 18:58:08+00:00	F	ER Sepsis Triage	NaN	NaN
72	2014-05-07 19:11:00+00:00	F	Leucocytes	NaN	18.8
73	2014-05-07 19:11:00+00:00	F	CRP	15.0	NaN
74	2014-05-07 19:11:00+00:00	F	LacticAcid	NaN	NaN
75	2014-05-07 22:01:28+00:00	F	IV Antibiotics	NaN	NaN
76	2014-05-07 22:01:28+00:00	F	IV Liquid	NaN	NaN
77	2014-05-07 22:03:00+00:00	F	Admission NC	NaN	NaN
78	2014-05-09 11:00:00+00:00	F	Release A	NaN	NaN

- Look at all the data of a single case *including event enrichments (introduced later)*
- Select the most interesting attributes in a suitable order for effective overview
- Go through the case event by event

pandas notation for empty

Case Inspection – Condensed View

Global Attributes Local Attributes Case Inspection Case Log Raw Events

Local attributes are collected per activity as *dictionaries*, i.e., set of (name: value) pairs

+ case:concept:name | is in | E X

Use filter to select a case.

Search...					Columns	Visualize	Explore	Export
case:concept:name	concept:name	time:timestamp	org:group	folded_data				
str unique: 1	str unique: 8	datetime64[us, Europe/Amsterdam]  Jan 15, 2015-Jan 15, 2015	str unique: 3	object (nulls: 4) unique:				
E	ER Registration	2015-01-15 20:14:58+01:00	A	{"DisfuncOrg":false,"DiagnosticUrinarySediment"				
E	ER Triage	2015-01-15 20:16:03+01:00	C	None				
E	ER Sepsis Triage	2015-01-15 20:17:15+01:00	A	None				
E	IV Liquid	2015-01-15 20:30:32+01:00	A	None				
E	CRP	2015-01-15 20:31:00+01:00	B	{"CRP":265}				
E	Leucocytes	2015-01-15 20:31:00+01:00	B	{"Leucocytes":8.5}				
E	LacticAcid	2015-01-15 20:31:00+01:00	B	{"LacticAcid":2.8}				
E	IV Antibiotics	2015-01-15 20:35:59+01:00	A	None				
8 rows, 5 columns					100 / page	<< < Page 1 of 1 > >>	No selection	

Standard Exploratory Analysis

... applied to event logs

Univariate Summaries

Example: Sepsis log

Global Attributes Local Attributes Case Inspection Case Log Raw Events

Select one or more activities to view only the data attributes that are common to their schema

Select activities to see local attributes

☒ Hide attributes that are also case-level attributes

Table Bar Chart X +

Search...

Index0 int64	case:concept:name str unique: 1,012	concept:name str unique: 1	time:timestamp datetime64[us, Europe/Amsterdam]
1	A	Leucocytes	2014-10-22 12:00:00
10	A	Leucocytes	2014-10-24 12:00:00
11	A	Leucocytes	2014-10-26 12:00:00
13	A	Leucocytes	2014-10-28 12:00:00
16	A	Leucocytes	2014-10-30 12:00:00
17	A	Leucocytes	2014-10-31 12:00:00
20	A	Leucocytes	2014-11-02 12:00:00
26	B	Leucocytes	2014-12-21 12:00:00
27	C	Leucocytes	2014-02-06 12:00:00

ROWS | COLUMNS X

3,382 rows, 4 columns

Show all Hide all

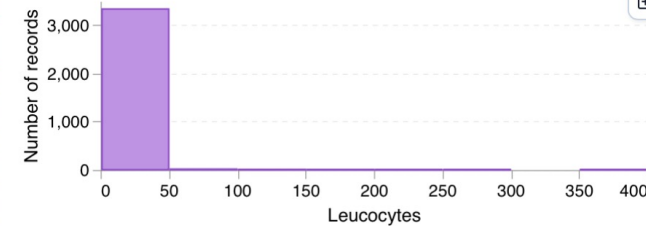
Search columns...

case:concept:name str

concept:name str

time.timestamp datetime64[us, Europe/Amsterdam]

Leucocytes float64



histogram

Total	3,382	Nulls	22
Unique	361	Min	0.2
Max	381.3	Std	15
Mean	12.95	Median	11
P5	3.1	P25	7.7
P75	15	P95	24.7

univariate summary
("boxplot" summary)

Histogram as Bar Chart with Binning

Global Attributes

Local Attributes

Case Inspection

Case Log

Raw Events

Select one or more activities to view only the data attributes that are common to their schemas:

Select activities to see local attributes

Leucocytes

☐ Hide attributes that are also case-level attributes

Table Bar Chart X +

⚠ Charts are not saved.

X-Axis

Leucocytes

Data Type

Sort

Bin step 4

Y-Axis

Count of records

Invert axis ☐

Color by

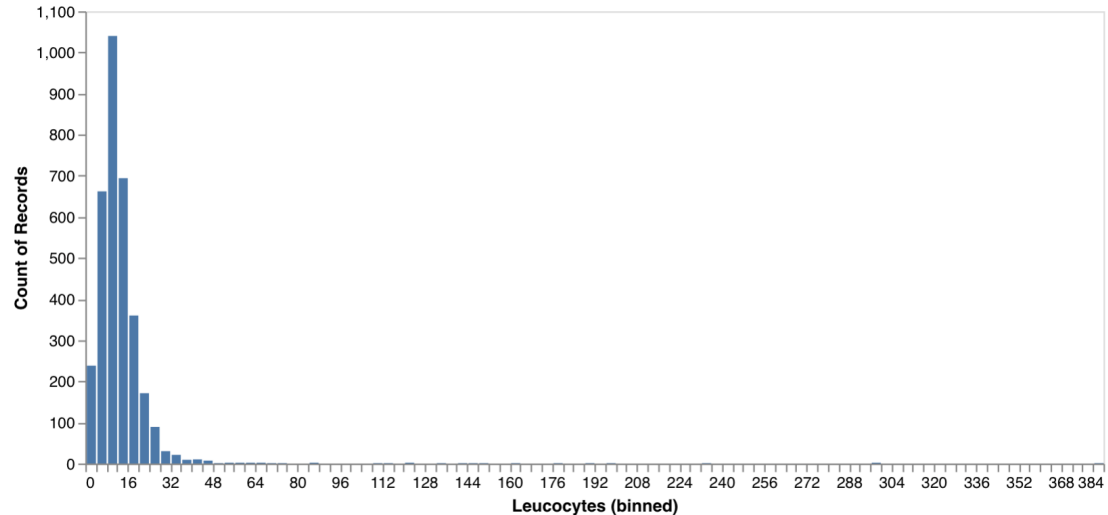
Select column

Faceting

Tooltips

Chart

Python code



Exercise

Compare the histograms for the attribute 'Age' as Local Attribute of activity 'ER Registration'

- as table summary on top of this column
- under Explore | Columns | Age
- under Visualize | Bar Chart (X-axis: Age; f: Bin (Choose a Bin Step), Y-Axis: Count of records)
- under Visualize | Bar Chart (X-axis: Age; f: None, Y-Axis: Count of records)

Histogram

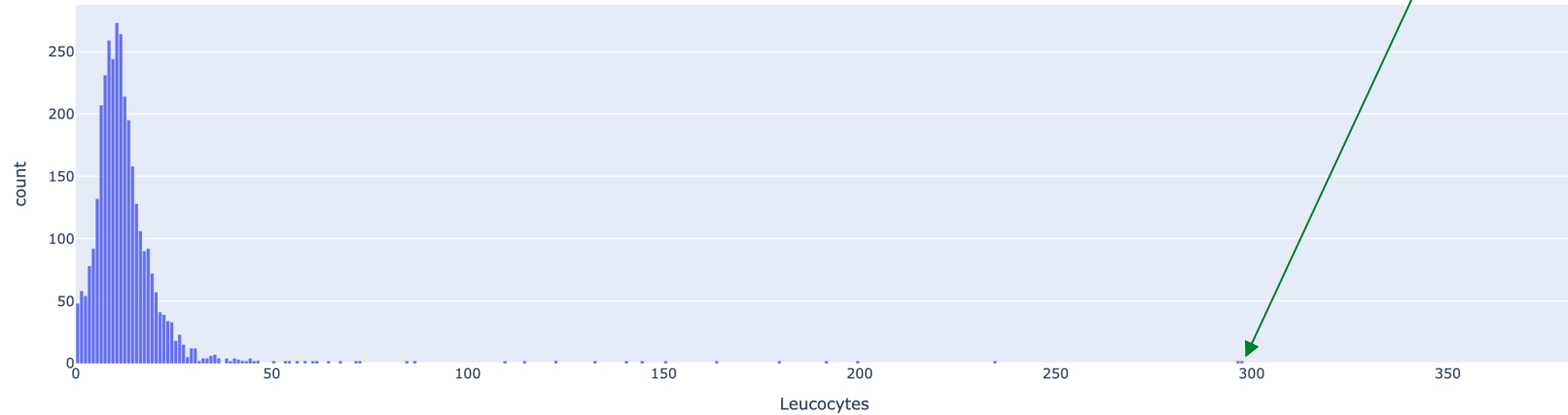
lots of interactivity built in can imply heavy memory load on browser

play with number of bins for optimal results

```
import plotly.express as px

fig = px.histogram(event_log, x='Leucocytes', nbins=500)
fig.update_layout(bargap=0.2)
fig.show()
```

outliers might be overlooked



Plotly histogram ref.

Log Scale

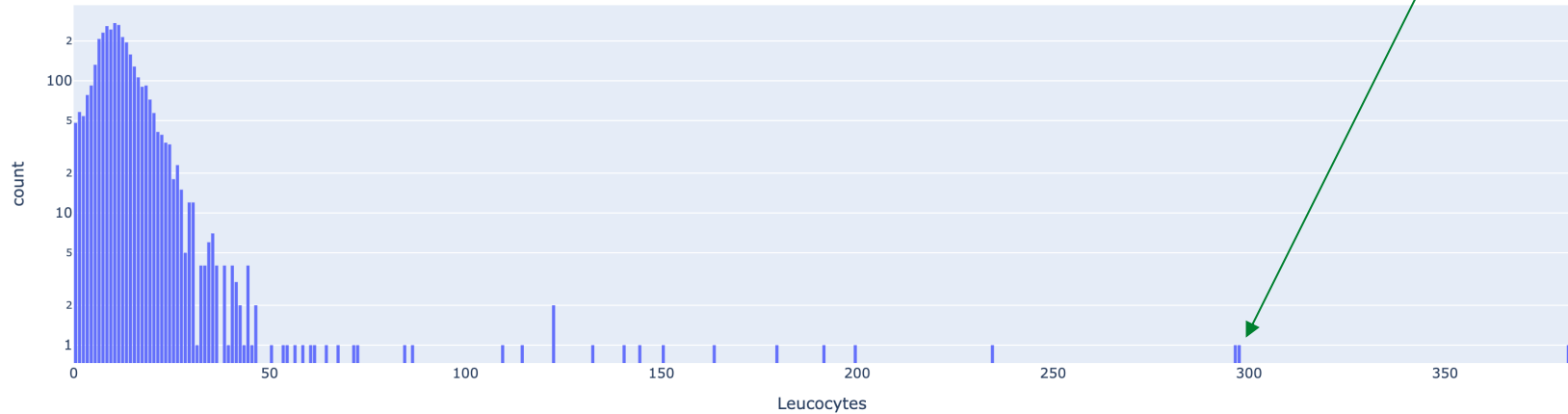
log scale for y-axis

play with number of bins for optimal results

```
import plotly.express as px

fig = px.histogram(event_log, x='Leucocytes', log_y=True, nbins=500)
fig.update_layout(bargap=0.2)
fig.show()
```

*better outlier
visibility,
but bars no longer
proportional*



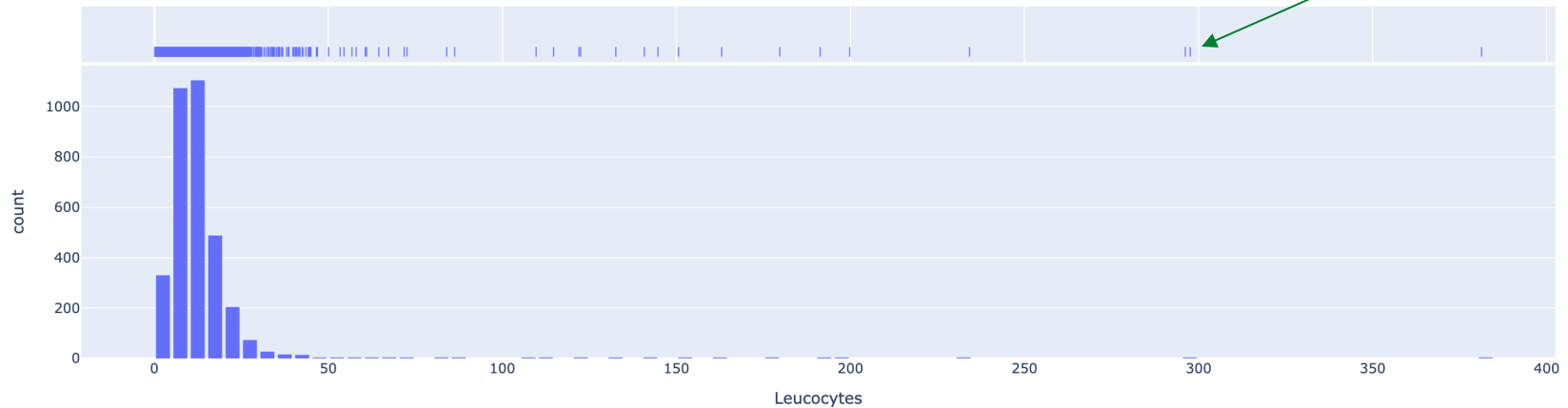
Plotly histogram ref.

Rug

```
fig = px.histogram(event_log, x='Leucocytes', nbins=100, marginal='rug')  
fig.update_layout(bargap=0.2)  
fig.show()
```

rug margin

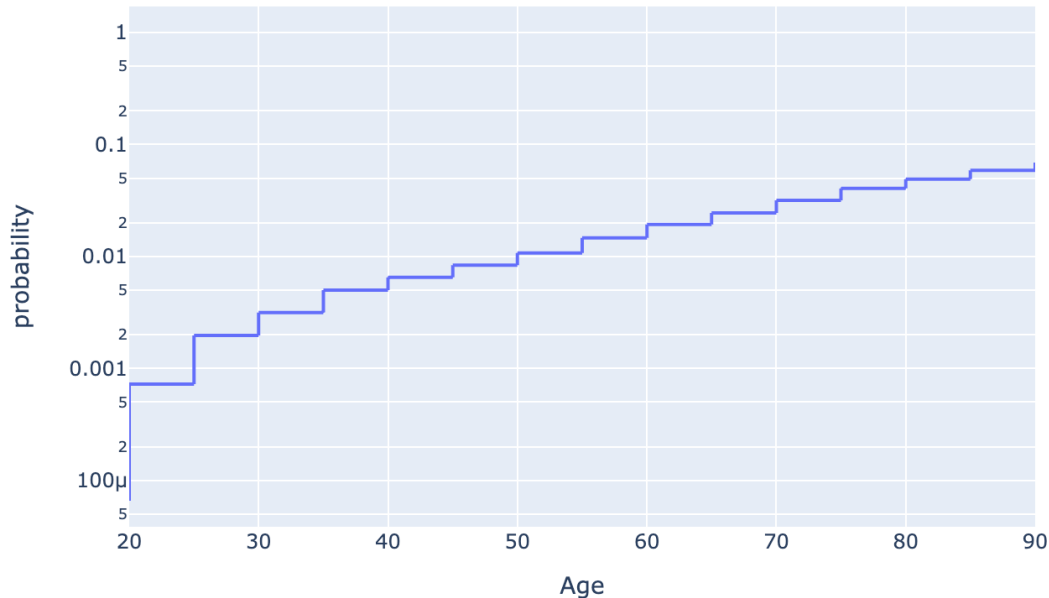
*better outlier
visibility,
with proportional
bars*



[Plotly histogram ref.](#)

Cumulative Distribution Plot

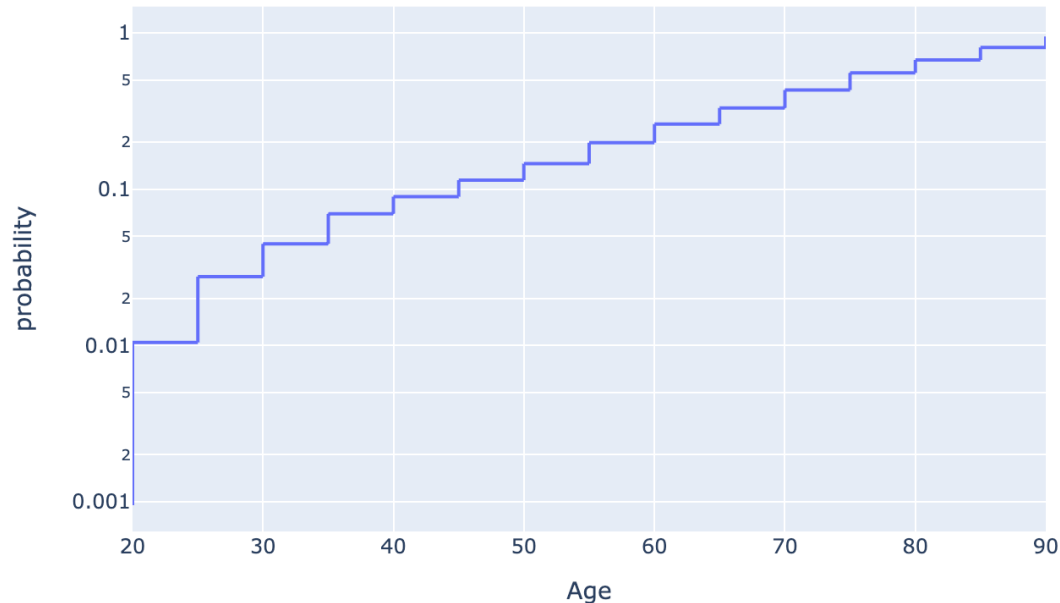
```
import plotly.express as px
fig = px.ecdf(event_log, x='Age', log_y=True, width=700)
fig.show()
```



Plots probability that Age is smaller or equal than a given value in the data set.

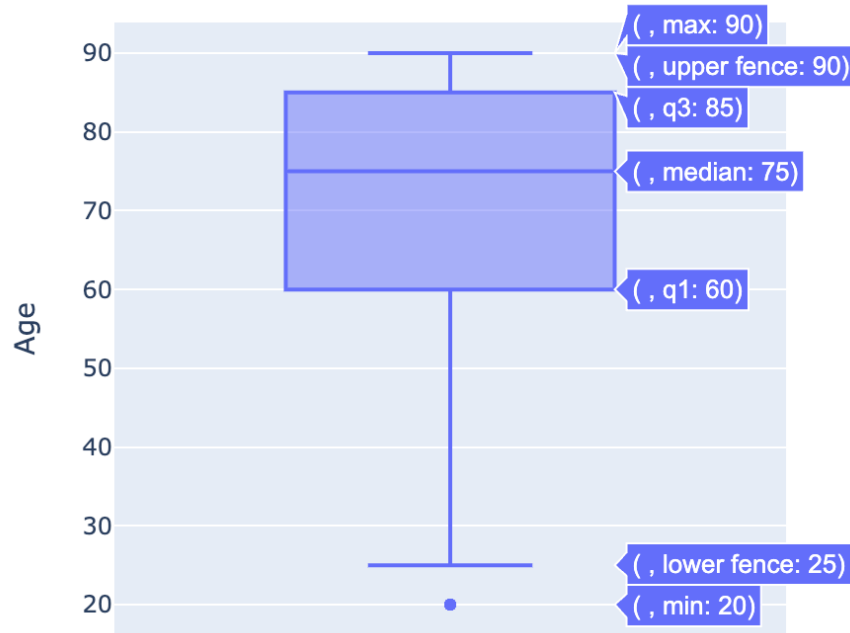
Cumulative Distribution Plot 2

```
filtered_event_log = event_log[event_log['concept:name']=='ER Registration']  
fig = px.ecdf(filtered_event_log, x='Age', log_y=True, width=700)  
fig.show()
```



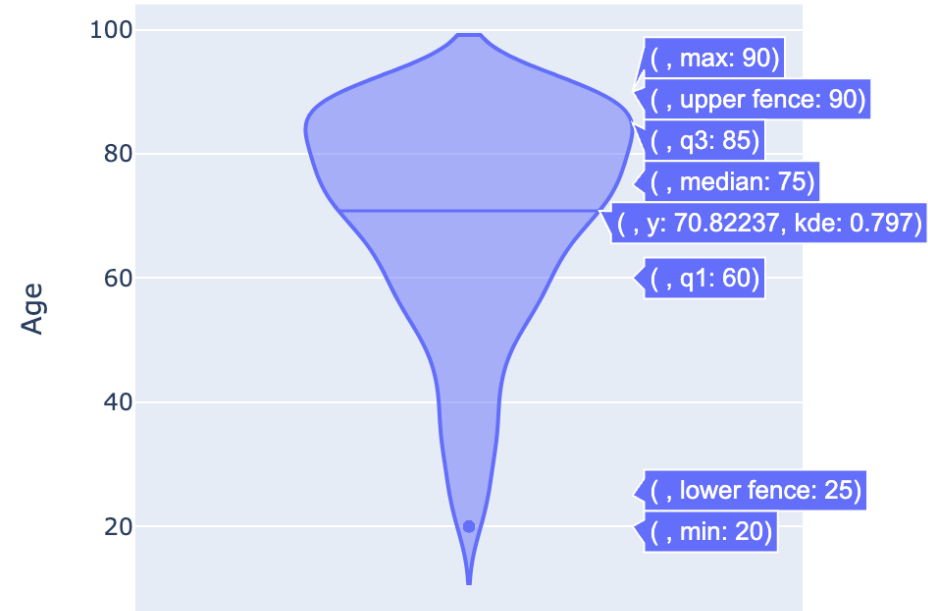
Plots probability that Age is smaller or equal than a given value in the 'ER Registration' activity

Box Plot and

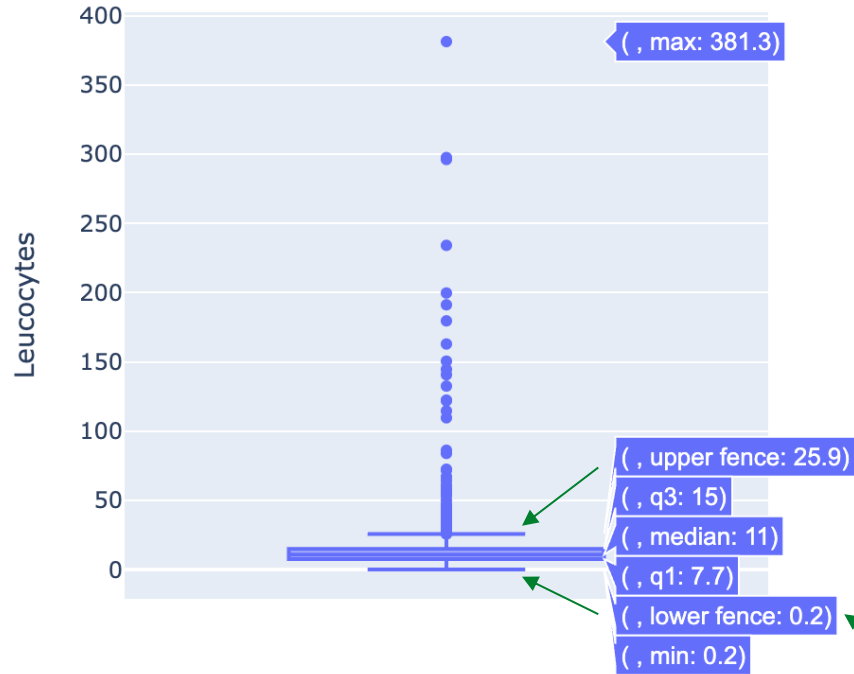


Violin Plot

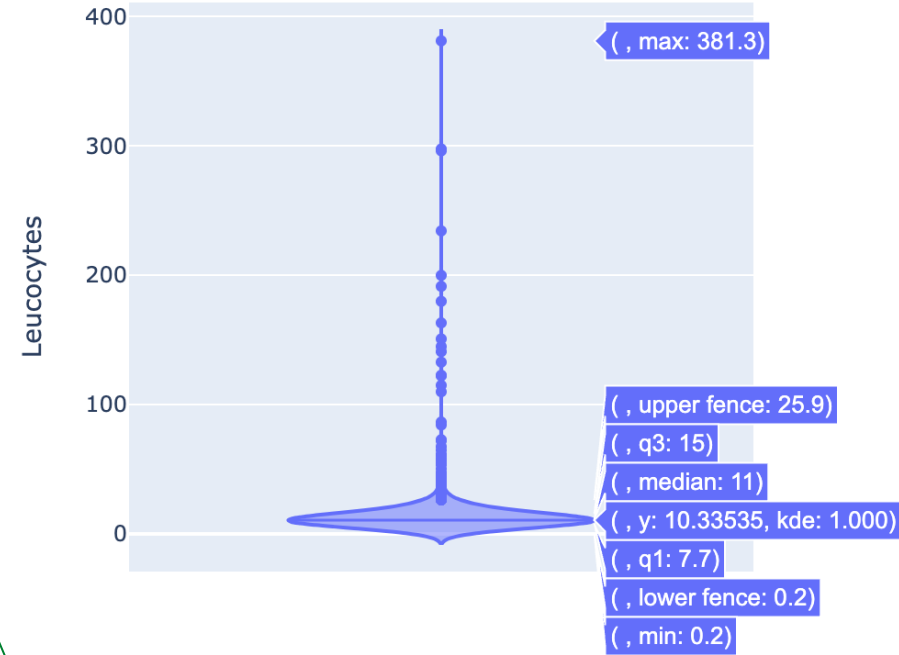
(don't use for attributes as discrete as Age)



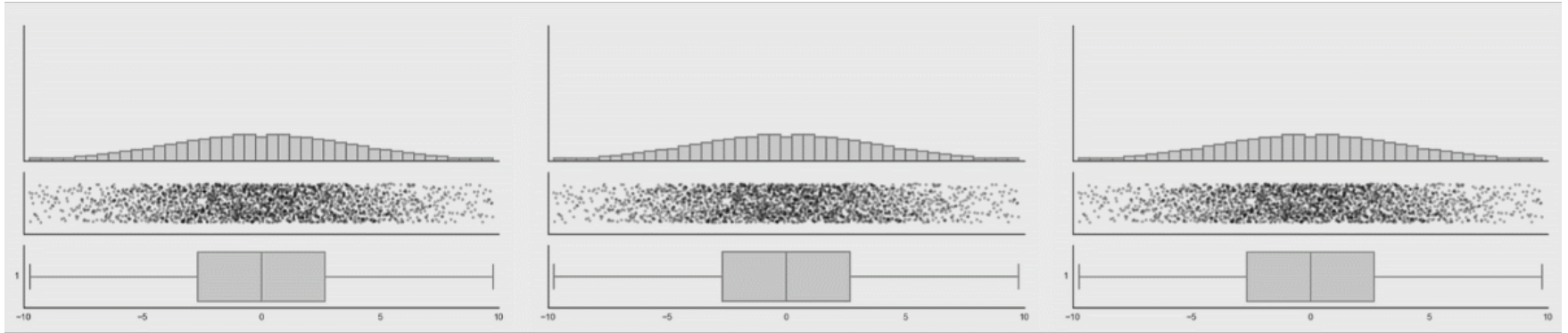
Outlier Display



Outlier fence



Boxplot vs. Histogram

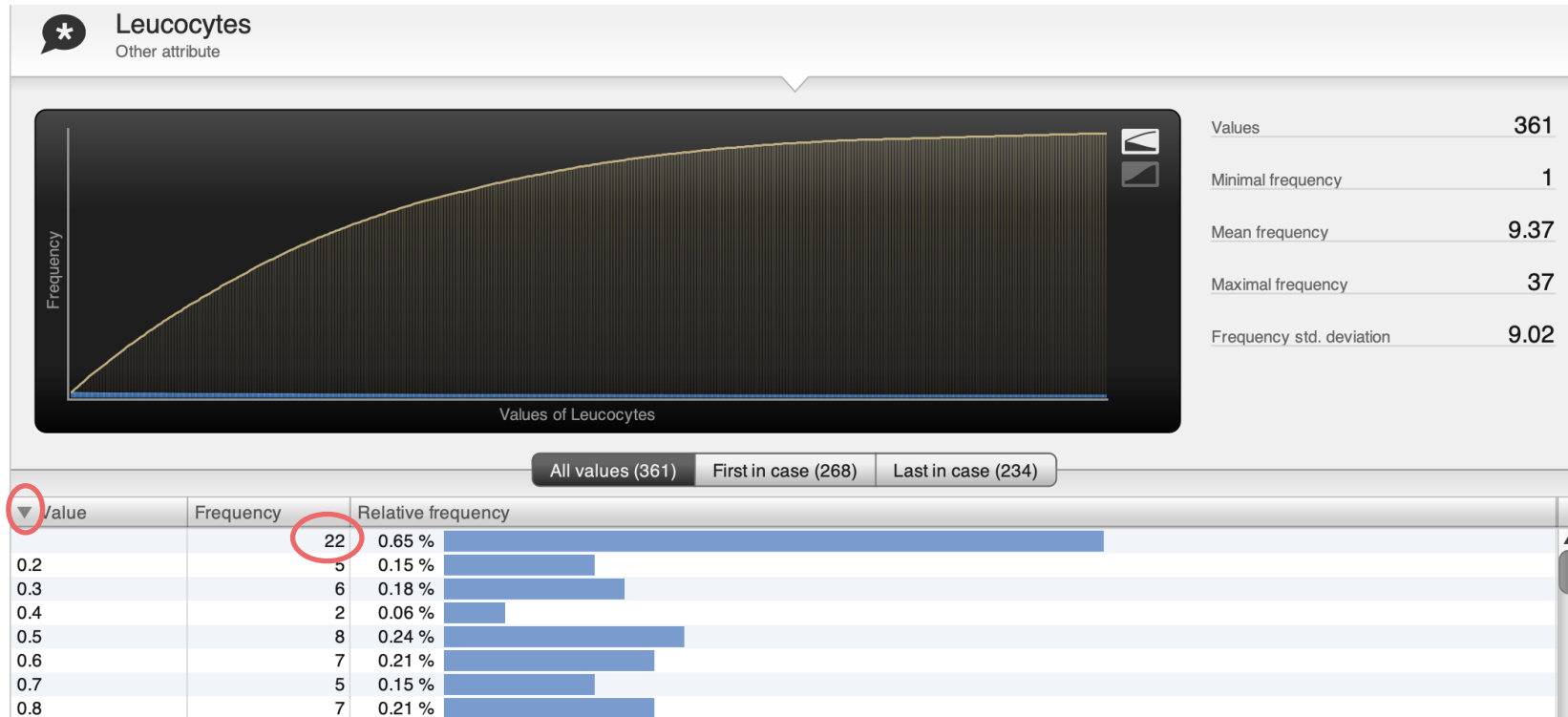


Source: [Autodesk](#)

Histogram in Disco

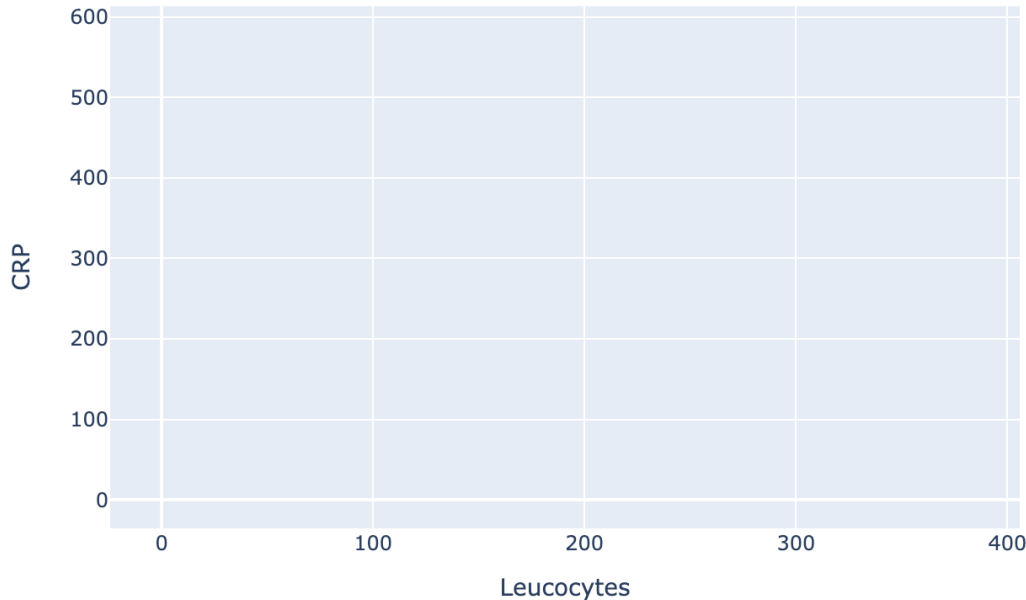


Histogram under Activity Filter in *Disco* (keep only *Leucocytes* activity)



Scatter Plot (first attempt)

```
fig = px.scatter(event_log, x='Leucocytes', y='CRP', width=700)  
fig.show()
```



Scatter plot show the relationship between two log attributes ...

The Case Log

= aggregate features of cases into a separate table

Grouping (e.g., group events into cases)

```
grouping = event_log.groupby(CASE_ID)
# for name, group in grouping:
for i, (name, group) in enumerate(grouping):
    if i == 5:
        break
    print('Case', name)
    display(group[[CASE_ID, ACTIVITY]])
```

Case AA

	case:concept:name	concept:name
297	AA	ER Registration
298	AA	ER Triage
299	AA	ER Sepsis Triage
300	AA	Leucocytes
301	AA	LacticAcid
302	AA	CRP
303	AA	IV Liquid
304	AA	IV Antibiotics

8 rows x 2 cols 10 ▾ per page

Case A

	case:concept:name	concept:name
0	A	ER Registration
1	A	Leucocytes
2	A	CRP
3	A	LacticAcid
4	A	ER Triage
5	A	ER Sepsis Triage
6	A	IV Liquid
7	A	IV Antibiotics
8	A	Admission NC
9	A	CRP
10	A	Leucocytes
11	A	Leucocytes
12	A	CRP
13	A	Leucocytes
14	A	CRP
15	A	CRP
16	A	Leucocytes
17	A	Leucocytes
18	A	CRP
19	A	CRP
20	A	Leucocytes
21	A	Release A

22 rows x 2 cols 25 ▾ per page

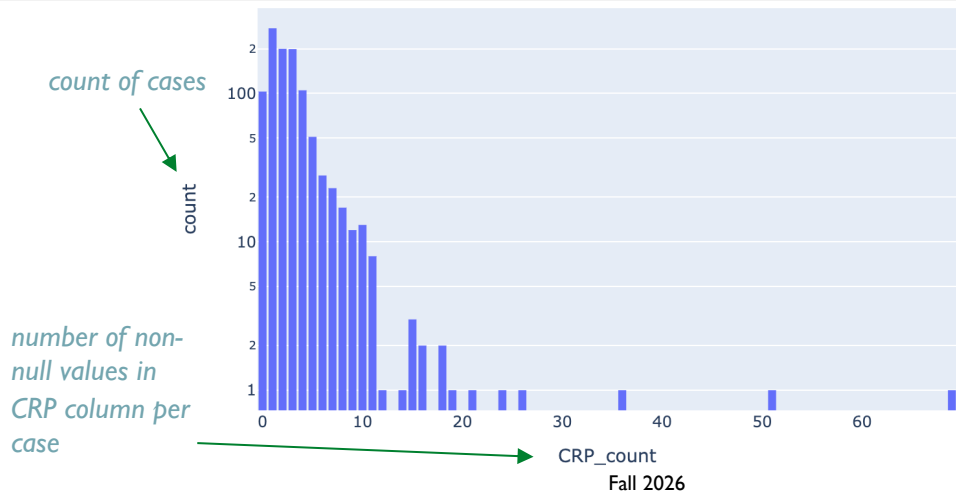
Case Aggregation

```
df = event_log.groupby(CASE_ID, as_index=False)['CRP'].count()
df = df.rename(columns= {'CRP': 'CRP_count'})
df
```

```
fig = px.histogram(df, x='CRP_count', nbins=100, log_y=True, width=700)
fig.update_layout(bargap=0.2)
fig.show()
```

case id →

	case:concept:name	CRP_count
0	A	7
1	AA	1
2	AAA	1
3	AB	1
4	ABA	4
...	...	...



The Case Log

```
cases = event_log.groupby(CASE_ID)
case_log = cases.agg(
    start_time=('time:timestamp', 'first'),
    end_time=('time:timestamp', 'last'),
    no_of_events=('concept:name', 'count'))
```

case id as index of the case log

aggregation operator

	start_time	end_time	no_of_events
case:concept:name			
A	2014-10-22 11:15:41+00:00	2014-11-02 15:15:00+00:00	22
AA	2014-12-03 09:06:44+00:00	2014-12-03 14:28:01+00:00	8
AAA	2014-11-19 03:16:21+00:00	2014-11-28 16:15:17+00:00	11
AB	2014-02-16 09:55:43+00:00	2014-02-16 13:52:06+00:00	8
ABA	2014-10-12 11:22:24+00:00	2014-10-18 16:15:00+00:00	17
...	...	...	...

We compile case level information into a separate dataframe, the *case log*, to

- simplify creation of case level visualization
- simplify testing of *enrichments* (cf. later)
- make data representation more efficient

Some Group Aggregation Operators

Finde more operators [here](#).

Aggregation	Description
<code>count()</code>	Total number of items, excluding missing values
<code>first(), last()</code>	First and last item within each group
<code>mean(), median()</code>	Mean and median
<code>min(), max()</code>	Minimum and maximum
<code>std(), var()</code>	Standard deviation and variance
<code>mad()</code>	Mean absolute deviation
<code>prod()</code>	Product of all items
<code>sum()</code>	Sum of all items

Exercise: *grouping_tutorial_sepsis.py*

Download from Canvas and open.

Learn/recap how to

- form groups
- aggregate in groups
- filter and enrich based on group properties

A Brief Tutorial for Pandas Grouping ...

... in the context of process mining

Introduction

We use again the Sepsis event log.

```
1 file_name = '../data/SepsisCases2020EventLog.xes'
2 event_log = pm4py.read_xes(file_name)
3
4 event_log
```

Search...

Columns

Visualize

Explore

Export

SIRSCriteria2OrMore

object (nulls: 14164)
unique:

Oligurie

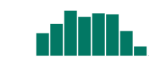
object (nulls: 14164)
unique:

SIRSCritTachypnea

object (nulls: 14164)
unique:

time.timestamp

datetime64[us, UTC]



Nov 7, 2013-Jun 5, 2015

Age

float64 (nulls: 1)



Case Attribute = Attribute such that there is at most one unique value per case

Global Attributes Local Attributes Case Inspection **Case Log** Raw Events

case attributes automatically added to case log

Q Search... Columns Visualize Explore Export

case:concept:name str	start_time datetime64[us, Europe/Amsterdam]  Nov 7, 2013-Feb 26, 2015	end_time datetime64[us, Europe/Amsterdam]  Nov 9, 2013-Jun 5, 2015	no_of_events int64  3 185	DisfuncOrg object unique:	DiagnosticUrinarySediment object unique:	DiagnosticLiquo object unique:
A	2014-10-22 11:15:41+02:00	2014-11-02 15:15:00+01:00	22	True	True	False
AA	2014-12-03 09:06:44+01:00	2014-12-03 14:28:01+01:00	8	False	True	False
AAA	2014-11-19 03:16:21+01:00	2014-11-28 16:15:17+01:00	11	False	False	False
AB	2014-02-16 09:55:43+01:00	2014-02-16 13:52:06+01:00	8	False	True	False
ABA	2014-10-12 11:22:24+02:00	2014-10-18 16:15:00+02:00	17	False	True	False
AC	2014-09-24 15:39:13+02:00	2014-12-02 18:47:17+01:00	13	False	True	False
ACA	2014-09-22 10:00:00+02:00	2014-09-22 10:45:56+02:00	8	False	False	False
AD	2013-12-19 09:04:38+01:00	2014-05-04 18:34:00+02:00	29	False	True	False
ADA	2014-06-21 07:14:16+02:00	2014-07-06 20:58:00+02:00	24	False	True	False
AE	2014-03-11 08:41:47+01:00	2014-07-23 17:38:24+02:00	18	False	False	False

1,050 rows, 27 columns 10 / page << < Page 1 of 105 > >> No selection

Scatter Plot

Global Attributes Local Attributes Case Inspection **Case Log** Raw Events

Table Scatter Chart X +

Scatter

Chart

Python code

Data

Style

Charts are not saved.

X-Axis

Age

None

Data Type

Number

Y-Axis

no_of_events

None

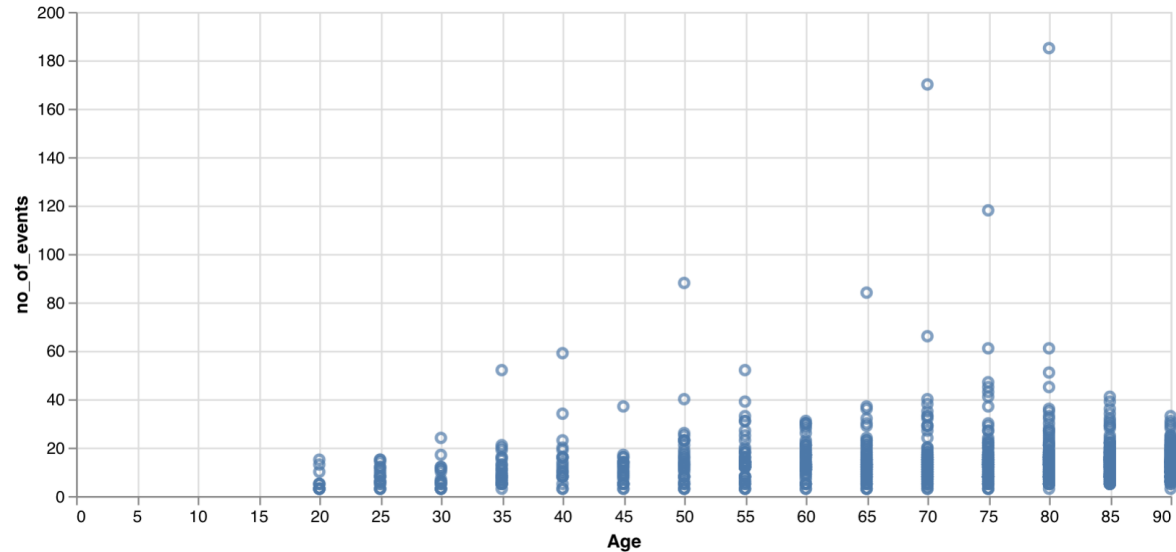
Data Type

Number

Invert axis

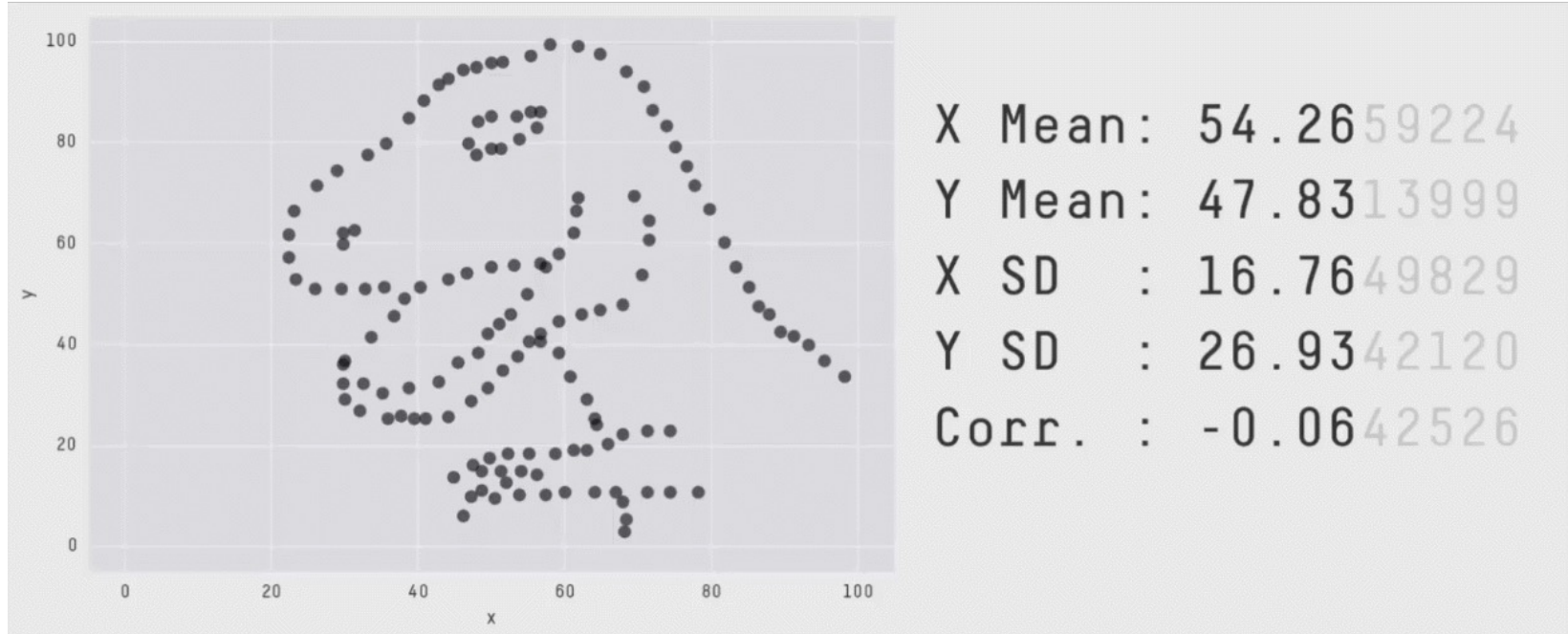
Color by

Select column



Faceting

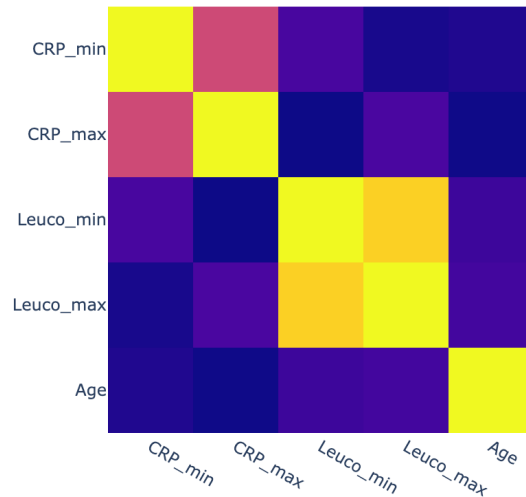
Scatter Plot vs. Summary (cf. [Anscombe's Quartet](#))



Source: [Autodesk](#)

Correlation Matrix

```
cor = df.corr(numeric_only=True)
fig = px.imshow(cor, width=600)
fig.show()
```



- correlation coefficient (Pearson) excluding null values for each pair of numeric case log features

← Exercise (*sepsis_v1.ipynb*)

Log (Attribute) Enrichment

= add *attributes*, (a.k.a. *features* or *columns*), to the log, typically by extracting and combining information from existing attributes.

Log Enrichment

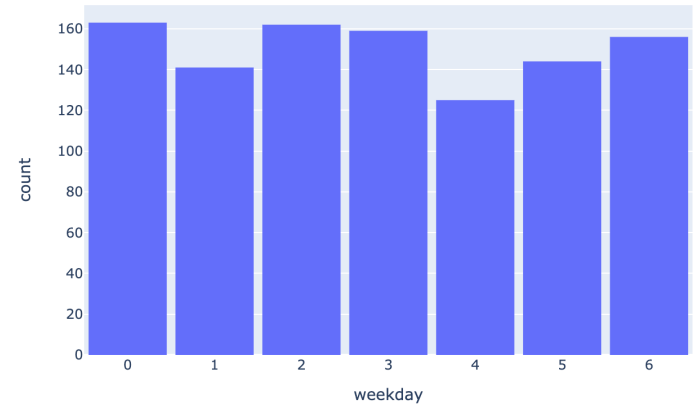
1. Event log enrichment
 - Univariate map
 - Multivariate map
 - Sequential aggregation
2. Case log enrichment

Univariate Map (= Create an attribute from a single existing attribute)

```
event_log['weekday'] = event_log['time:timestamp'].apply(lambda x: x.weekday())  
display(event_log[['case:concept:name', 'time:timestamp', 'concept:name', 'weekday']])
```

	case:concept:name	time:timestamp	concept:name	weekday
0	A	2014-10-22 11:15:41+00:00	ER Registration	2
1	A	2014-10-22 11:27:00+00:00	Leucocytes	2
2	A	2014-10-22 11:27:00+00:00	CRP	2
3	A	2014-10-22 11:27:00+00:00	LacticAcid	2
4	A	2014-10-22 11:33:37+00:00	ER Triage	2
...	...	...	...	...

- Example: parse out weekday from timestamp to understand correlation between weekday and other attribute(s)



Count of ER Registration events per weekday

Exercise (*sepsis_v1.ipynb*)

Multivariate Map (= Create an attribute from multiple existing attributes)

```
event_log = pd.read_csv('Purchasing-Example.csv', parse_dates=['Start Timestamp', 'Complete Timestamp'])

event_log['duration'] = event_log['Complete Timestamp'] - event_log['Start Timestamp']

event_log['duration_secs'] = event_log.apply(
    lambda row: (row['Complete Timestamp'] - row['Start Timestamp']).seconds,
    axis=1)

display(event_log[['Start Timestamp', 'Complete Timestamp', 'Activity', 'duration', 'duration_secs']])
```

	Start Timestamp	Complete Timestamp	Activity	duration	duration_secs
0	2011-02-16 14:31:00	2011-02-16 15:23:00	Create Purchase Requisition	0 days 00:52:00	3120
1	2011-02-17 09:34:00	2011-02-17 09:40:00	Analyze Purchase Requisition	0 days 00:06:00	360
2	2011-02-17 21:29:00	2011-02-17 21:52:00	Amend Purchase Requisition	0 days 00:23:00	1380

datetime64[ns] *timedelta64[ns]* *int64*

- *apply (axis=1)* can take functions over rows

Sequential Aggregation

```
from event_enricher import track_variables

tracking_list = [('CRP', 'latest'), (ACTIVITY, 'count')]
event_log = track_variables(event_log, tracking_list)
df = event_log[[CASE_ID, ACTIVITY, 'CRP', 'CRP::latest', 'concept:name::count']]
df.head(50)
```

	case:concept:name	concept:name	CRP	CRP::latest	concept:name::inc
0	A	ER Registration	NaN	NaN	1
1	A	Leucocytes	NaN	NaN	2
2	A	CRP	21.0	21.0	3
3	A	LacticAcid	NaN	21.0	4
4	A	ER Triage	NaN	21.0	5
5	A	ER Sepsis Triage	NaN	21.0	6
6	A	IV Liquid	NaN	21.0	7
7	A	IV Antibiotics	NaN	21.0	8

8	A	Admission NC	NaN	21.0	9
9	A	CRP	109.0	109.0	10
10	A	Leucocytes	NaN	109.0	11
11	A	Leucocytes	NaN	109.0	12
12	A	CRP	47.0	47.0	13
13	A	Leucocytes	NaN	47.0	14
14	A	CRP	15.0	15.0	15
15	A	CRP	9.0	9.0	16
16	A	Leucocytes	NaN	9.0	17

at the time of administering IV Antibiotics, the latest CRP value was 21.0

Sequential Aggregation

- specifically useful for event logs
- think of a case variable that is updated or not updated by each event of a case and has a value before and after each event
- assumes that timestamps monotonically increase for each case in the log
 - this can be verified as follows

```
event_log.groupby('case:concept:name')[COMPLETION_TIME].is_monotonic_increasing.all()
```

Always test your enrichments (here: with a random case)

```
import random

random_case_id = random.choice(event_log[CASE_ID].unique())
case_data = event_log[event_log[CASE_ID] == random_case_id]
case_data[columns_to_show]
```

Recommendation:

If an enrichment is composed of multiple operations, test one operation at a time.

case:concept:name	concept:name	# CRP	# CRP::latest	# CRP::max-latest
DFA	ER Registration	Missing value	Missing value	Missing value
DFA	ER Triage	Missing value	Missing value	Missing value
DFA	ER Sepsis Triage	Missing value	Missing value	Missing value
DFA	Leucocytes	Missing value	Missing value	Missing value
DFA	CRP	13.0	13.0	0.0
DFA	Admission NC	Missing value	13.0	0.0
DFA	Admission NC	Missing value	13.0	0.0
DFA	Leucocytes	Missing value	13.0	0.0
DFA	CRP	7.0	7.0	6.0
DFA	Release A	Missing value	7.0	6.0

Sequential Aggregation: Available Operators

Operator	Description	Alternative implementation provided by <i>pandas</i>
col::latest	latest value of col in case	<code>event_log['col::latest'] = event_log.groupby('case_id')[col].ffill()</code>
col::count	cumulative count of col in case	<code>... [col].cumcount()</code> (<i>counting starts at 0</i>)
col::min	cumulative min of col in case	<code>... [col].cummin().ffill()</code>
col::max	cumulative max of col in case	<code>... [col].cummax().ffill()</code>
col::sum	cumulative sum of col in case	<code>... [col].cumsum().ffill()</code> (<i>initializes the sum with 0 not with NaN</i>)
col::append	cumulative list of col values	

Be careful to observe the behavior of the implementation w.r.t. the treatment of *NaN* and whether counting starts at 0 or 1.

Exercise Grouping (in grouping_tutorial_sepsis.py)

1. Create a table that shows, for each case of the event log,
 - a) the first entry of the column *org:group*. What is the value for case *CJ*?
 - b) the mean for each of the following columns: *CRP*, *Leucocytes*, *LacticAcid*; What are the values for case *CJ*?
 - c) the number of events
 - d) the number of *CRP* events
 - e) the case start time, case end time, and case duration
2. Create a table that shows, for each date of the log, the number of *CRP* events per day.
3. Create a table that shows, for each activity of the log, the earliest time that activity occurs in the log.

Exercise Enrichment (in sepsis_v1.py)

1. Track for each event of the Sepsis log,
 - a) the latest *Leucocytes* value in the case
 - b) the date of the latest CRP activity in the case
 - c) the number of CRP events in the case so far
 - d) the time since *Admission IC* in that case if applicable, NaN otherwise

Verification (for pre-existing enrichment)

True becomes 1, False becomes 0

SIRS



```
event_log['SIRSCriteria20rMore_expected'] = (event_log['SIRSCritHeartRate'] + event_log['SIRSCritLeucos']  
      + event_log['SIRSCritTachypnea'] + event_log['SIRSCritTemperature']) >= 2  
diff_log = event_log[event_log['SIRSCriteria20rMore_expected'] != event_log['SIRSCriteria20rMore']]  
  
selected_columns = sorted([ col for col in event_log.columns if col.startswith('SIRS')])  
display(diff_log[diff_log[ACTIVITY]=='ER Registration'][selected_columns])
```

 SIRSCritHeartRate	 SIRSCritLeucos	 SIRSCritTachypnea	 SIRSCritTemperature	 SIRSCriteria20rMore	 SIRSCriteria20rMore_expected
True	False	False	True	False	True
False	False	True	False	False	True
True	False	False	True	False	True
True	False	True	False	False	True
True	False	False	True	False	True
False	False	False	False	True	False

37 rows x 6 cols

Case Log Enrichment (*just as event log enrichment*)

```
tracking_list = [('concept:name', 'append')]
event_log = track_variables(event_log, tracking_list)
cases = event_log.groupby('case:concept:name')
case_log['seq_variant'] = cases['concept:name::append'].last()
```

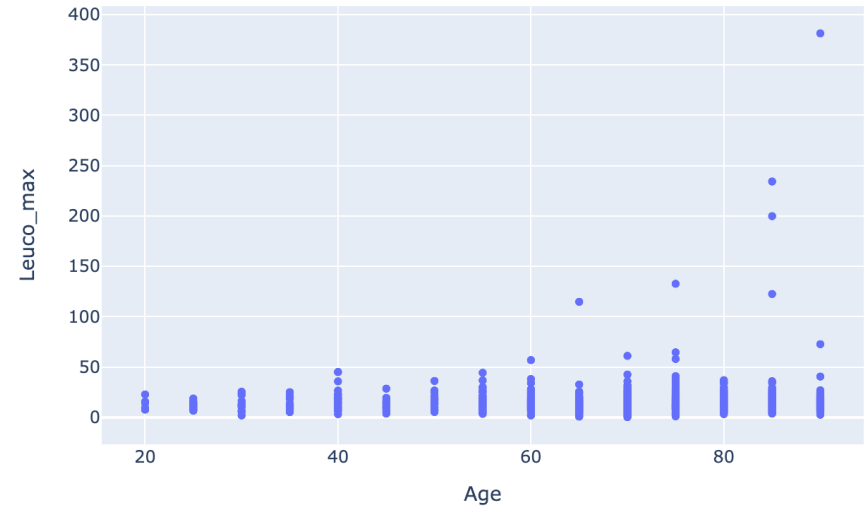
case:concept:name	seq_variant
A	['ER Registration', 'Leucocytes', 'CRP', 'LacticAcid', 'ER Triage', 'ER Sepsis Triage', 'IV Liquid', 'IV Antibiotics', 'Admission NC', 'CRP', 'Leucocytes', 'Leucocytes']
AA	['ER Registration', 'ER Triage', 'ER Sepsis Triage', 'Leucocytes', 'LacticAcid', 'CRP', 'IV Liquid', 'IV Antibiotics']
AAA	['ER Registration', 'ER Triage', 'ER Sepsis Triage', 'IV Liquid', 'Leucocytes', 'CRP', 'LacticAcid', 'IV Antibiotics', 'Admission NC', 'Release A', 'Return ER']
AB	['ER Registration', 'ER Triage', 'ER Sepsis Triage', 'CRP', 'LacticAcid', 'Leucocytes', 'IV Liquid', 'IV Antibiotics']
ABA	['ER Registration', 'ER Triage', 'ER Sepsis Triage', 'IV Liquid', 'LacticAcid', 'Leucocytes', 'CRP', 'IV Antibiotics', 'Admission NC', 'Leucocytes', 'CRP', 'Leucocytes']
AC	['ER Registration', 'ER Triage', 'ER Sepsis Triage', 'Leucocytes', 'LacticAcid', 'CRP', 'IV Liquid', 'IV Antibiotics', 'Admission NC', 'CRP', 'Leucocytes', 'Release A']
ACA	['ER Registration', 'ER Triage', 'ER Sepsis Triage', 'CRP', 'LacticAcid', 'Leucocytes', 'IV Liquid', 'IV Antibiotics']

Scatter Plot on Case Log

we verified that this is unique

```
cases = event_log.groupby(CASE_ID)
df = cases.agg(Leuco_max=('Leucocytes', 'max'), Age=('Age', 'max'))
fig = px.scatter(df, x='Age', y='Leuco_max', width=700)
fig.show()
```

	Leuco_max	Age
case:concept:name		
A	13.0	85.0
AA	8.6	75.0
AAA	20.8	60.0
AB	15.7	90.0
ABA	5.2	75.0
...	...	...



Code Generation Issues

- Did you precisely and unambiguously express your intent?
- Was the code generation correct?
- Complication: For a given task, there are multiple ways to code it, not every way is equally *readable* to you.

Coding Recommendations 1

- The code provided in the slides and notebooks is optimized for readability.
 - Recommendation: **Use the provided code whenever possible**. This increases your chances that your code is correct.
 - Often, only minor modifications are needed.
 - There are no extra points for code originality. *(Of course, we appreciate if you come up with something better, in particular if you find better visualizations for a given task.)*
- **Invest into understanding the given** (i.e., provided or generated) **code**
 - Ask your digital tutor (*Claude, Copilot, ChatGPT, ...*) to help you understand.
 - Discuss alternative ways to write the code until you have code that you understand best.
 - This helps you develop a progressively more accurate understanding of what correct code looks like.

Coding Recommendations 2

- Be aware of imprecision of standard floating point arithmetic
 - this matters when using thresholds
- Use fixed precision and rounding or integers for discrete numbers such as money

`0.3 - 0.2`

✓ 0.0s

`0.09999999999999998`

`0.3 - 0.2 >= 0.1`

✓ 0.0s

`False`

`round(0.3-0.2,1)`

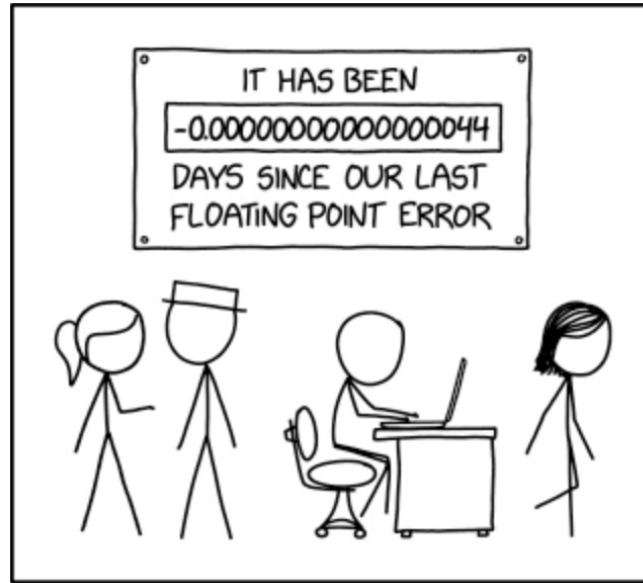
✓ 0.0s

`0.1`

`round(0.3-0.2,1) >= 0.1`

✓ 0.0s

`True`



<http://xkcd.com/3228>

Further Reading

Wilke: [Fundamentals of Data Visualization](#), O'Reilly

Chapter on [Exploratory Data Analysis](#)

Assignment 1: Notice

Keep in mind that you are studying a real-world event log, not an academically polished exercise log. This means:

- data might be missing or inconsistent
- events might not always exactly mean what you initially assume
- we do not know all details of the data collection and event log preparation process
- the given description (in Session 1) is what the data providers have published

This of course does not mean that every unexpected observation can be attributed to dirty log data. In fact, most strange observations will make sense in the end.

Assignment 1: Tasks from Session 2, Part 1

(Schema Analysis, Case Inspection)

1. Which granularity do the timestamps of the log have? *[Code welcome but not required.]*
2. Inspect the schemas of all activities. *[Code welcome but not required.]*
 - a) Which activity schemas have attributes with missing values? How many events are affected for each such attribute?
 - b) Which non-global attributes are shared, i.e., local to more than one activity schema? Classify each such attribute that is also numerical as either cumulative or incremental.
3. Select a case of the log with more than three events. Write a short bullet point for each event that summarizes what happened paying attention to the timestamps and all attributes. *[Code welcome but not required.]*

Assignment 1: Tasks from Session 2 Part 2 (*Event enrichment*)

4. Add an attribute to the event log that represents, for each case, the cumulative sum of payment amounts after each event.
 - a) Select and inspect a case (including this new enrichment) to verify that the enrichment was computed correctly.
 - b) Recall that the already existing attribute *totalPaymentAmount* also claims to represent the cumulative sum of payment amounts. Verify that *totalPaymentAmount* was computed correctly by comparing it with the attribute you just added. If you find a deviation, quantify the deviation and inspect and interpret a deviating case.
5. Add an attribute *outstanding_amount* to the event log that represents, for each case, the amount that still has to be paid by the offender after each event. Recall that the offender has to pay the *initial fine amount*, the *penalty* if added and any *expenses* that are recorded. Define *outstanding_amount* and any preliminary attributes as event enrichments.
 - a) Select and inspect a case (including this new enrichment) that the enrichment was computed correctly.
 - b) What is the number of events that have an outstanding amount > 0 ?

Assignment 1: Tasks from Session 2 Part 3 (*Case enrichment*)

5. Create an initial case log that includes all *case attributes*. Add an attribute to the case log that represents the *initial fine amount* (i.e., the *amount* associated with the *Create Fine* event, or equivalently, the first value of *amount* of each case).
 - a) Formulate a hypothesis about the distribution of the initial fine amount.
 - b) Create a suitable visualization of the distribution of the initial fine amount. Describe briefly two interesting observations about the distribution.
6. Create a visualization relating initial fine amount and the *final fine amount* of each case. The *final fine amount* can be defined as the last value of the attribute *amount* for each case. Briefly describe and interpret the relationship that you see.
7. Find at least one noteworthy fact about the distribution (or a relationship) for any of the following attributes: *org:resource*, *dismissal*, *article*, *points*, or *expense*.